

AI-Assisted Python

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-03-31

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git collaboration	24/3	N	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: The Python ecosystem	21/4	Y	RZ D8
10: cont. L9 & project feedback	28/4	Y	RZ D8
11: Testing and debugging	5/5	N	ML H41.1
12: OOP	12/5	Y	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment **today**. Break next week!

Important project dates

Keep these dates in mind for your project planning!

Milestone	Date	TODO
Project chosen	13/3	Done
Project Structured	27/3	Done
Implementing	31/3	Starting today — 8 weeks to code and refine
Project draft	15/5	Pre-submission of project for feedback
Project submission	29/5	Final submission of project on Git (tag)

A note on assignments

Your first assignment is available today. To achieve full marks you have to:

- Make an effort to solve it and turn it in (**by next lecture**)
- Provide feedback/self-reflection (**by next lecture**)
- Provide peer feedback to a classmate — this will be randomly assigned to you after next lecture, once this week's homework is turned in. You will have 1 week to complete it (**until the following lecture**)

You can turn in the homework and self-assessment directly on Moodle. Instructions are found there.

Today's learning objectives

1. Explore ways in which AI changes the way we learn to code
2. Get comfortable with Python fundamentals: variables, types, operators, conditionals, and loops
3. Read and understand short Python scripts
4. Teaser into data structures

Contents

AI and Python	6
Python Fundamentals	11
Control Flow	26
Exceptions	35
Loops	43
Recap	54

AI and Python

Does syntax still matter?

Before we write a single line of Python — there is an ongoing debate about this.

“The syntax barrier is disintegrating”

- AI retrieves syntax instantly — no memorisation needed
- Natural language is becoming the real interface
- The barrier for coding has never been lower
- Focus on problem-solving

“Experts more important than ever”

- Code is written for humans to read, not just machines to run
- Debugging code requires comprehension
- Natural language is too ambiguous
- Syntax knowledge becomes a **differentiator**

So where does that leave us?

AI can write syntactically correct code for you. What AI **cannot** do:

Understanding > memorising

So where does that leave us?

AI can write syntactically correct code for you. What AI **cannot** do:

- Decide **what** your program should do
- Know **which** data structure makes sense for your problem
- Tell you **why** the output is wrong conceptually
- Evaluate whether the generated code is correct for **your** use case
- Understand the **broader context** of your project and how the code fits into it

So where does that leave us?

AI can write syntactically correct code for you. What AI **cannot** do:

- Decide **what** your program should do
- Know **which** data structure makes sense for your problem
- Tell you **why** the output is wrong conceptually
- Evaluate whether the generated code is correct for **your** use case
- Understand the **broader context** of your project and how the code fits into it

It's becoming more important to **read and understand code**,
not memorise how to write it from scratch.

A — AI as explainer

Ask AI to explain a concept, show variations, or quiz you on what you just read.

Best for: building understanding

B — AI as pair programmer

Write code **with** AI inline. Use prompts to guide it, stay in control of the logic.

Best for: day-to-day coding

C — AI as starting point

Generate a solution, then read, dissect, and modify until you understand it.

Best for: learning new concepts

Python Fundamentals

Why Python?

Python is the dominant language for scientific computing and data analysis. It is:

- **Readable** — syntax reads almost like English
- **General-purpose** — from quick scripts to large applications
- **Well-supported** — massive ecosystem of libraries
- **Interpreted** — no compilation step, fast feedback loop
- **Great for AI support** — AI models are often trained on Python code, so they understand it well

Computer program components

On a very high level, all computer programs — regardless of programming language — consist of the following:

- **Values** — the raw data your program works with
- **Variables** — named containers that store values
- **Data types and data structures** — how values are categorised and organised
- **Instructions** — the logic that acts on values and variables

Today we will have a surface-level overview of what Python offers for each of these.

A **variable** in Python stores a **value** under a name.

```
name = "Alice" # A string
age = 30        # An integer
height = 1.75   # A float
is_student = True # A boolean
nothing = None  # The absence of a value
```

A **variable** in Python stores a **value** under a name.

```
name = "Alice" # A string
age = 30        # An integer
height = 1.75   # A float
is_student = True # A boolean
nothing = None  # The absence of a value
```

Python is **dynamically typed** — the variable types can change at runtime.

```
x = 10
x = "now a string" # valid in Python
```

Variable naming conventions

Python's style guide (**PEP 8**) gives clear rules for naming variables:

Rule	Good	Bad
Use snake_case	<code>student_name = "Alice"</code>	<code>StudentName = "Alice"</code>
Be descriptive	<code>temperature_c = 25.5</code>	<code>x = 25.5</code>
Constants in UPPER_CASE	<code>PI = 3.14159</code>	<code>pi = 3.14159</code>
Avoid abbreviations	<code>number_of_students = 50</code>	<code>numStuds = 50</code>
Use type hints	<code>lucky_number: int = 7</code>	<code>lucky_number = 7</code>

Python has reserved words called **keywords** — they define the language's syntax and cannot be used as variable names.

```
help("keywords") # run this in a Python terminal to see the full list
```



Note:

Keywords appear in a different colour in your editor. You'll learn most of them naturally over the course.

Here are some of the most common keywords you'll see in Python code:

Keyword	What it does
<code>if / elif / else</code>	conditional branching
<code>for / while</code>	loops
<code>def / return</code>	define and exit functions
<code>import / from / as</code>	bring in external modules
<code>True / False / None</code>	built-in values
<code>and / or / not</code>	logical operators
<code>in</code>	membership test — <code>"a" in "cat" → True</code>
<code>break / continue / pass</code>	loop control

```
5 + 2    # 7    – addition
5 - 2    # 3    – subtraction
5 * 2    # 10   – multiplication
5 / 2    # 2.5  – division (always returns float)
5 // 2   # 2    – floor division (integer result)
5 % 2    # 1    – modulo (remainder)
5 ** 2   # 25   – exponentiation
```

```
5 + 2    # 7    – addition
5 - 2    # 3    – subtraction
5 * 2    # 10   – multiplication
5 / 2    # 2.5  – division (always returns float)
5 // 2   # 2    – floor division (integer result)
5 % 2    # 1    – modulo (remainder)
5 ** 2   # 25   – exponentiation
```

Comparison operators — always return True or False

```
5 == 5   # True   – equal to
5 != 4   # True   – not equal to
5 > 3    # True   – greater than
5 <= 5   # True   – less than or equal to
```

Type	Example	Notes
int	42, -7	whole numbers
float	3.14, -0.5	decimal numbers
complex	3+2j, 1j	complex numbers
str	"hello", 'world'	text, single or double quotes
bool	True, False	note the capital letters
None	None	represents “no value”

You can check a variable's type with `type()`:

```
type(42)          # <class 'int'>
type("hello")    # <class 'str'>
type(True)       # <class 'bool'>
```


MiniTask: Variables and types

1. Define a few variables of different types — at least one `int`, one `float`, one `str`, and one `bool`
2. Use `type()` to check each variable's type — e.g., `type(my_var)`
3. Now delete one of your variables with `del`:
4. Try calling the deleted variable, e.g., using the built-in `print()` function.

When something goes wrong, Python prints a **traceback**.

```
Traceback (most recent call last):  
  File "script.py", line 4, in <module>  
    print(my_var)
```

When something goes wrong, Python prints a **traceback**.

```
Traceback (most recent call last):  
  File "script.py", line 4, in <module>  
    print(my_var)
```

1. **“Traceback (most recent call last)”** — Python lists every function call that led to the crash, innermost last
2. **File and line number** — tells you exactly where to look
3. **The offending line** — the code that triggered the error
4. **Exception type + message** — `NameError` is the **what**; `'my_var' is not defined` is the **why**

Reading a traceback

Common exception types you will encounter early on:

Exception	What it means	Typical cause
NameError	variable or function not defined	typo, or used before assignment
TypeError	wrong type for an operation	"2" + 2 — can't add str and int
ValueError	right type, but bad value	int("hello")
IndexError	index out of range	list has 3 items, you asked for [5]
SyntaxError	Python can't even parse the code	missing colon, unmatched bracket

Strings can use single, double, or triple quotes — choose consistently:

```
"It's me!"           # preferred: double quotes avoid escaping the apostrophe
'It\'s me!'          # works, but requires escaping
"""She said, "It's me!". """ # triple quotes for embedded quotes

path = r"C:\Users\Alice\file.txt"      # raw string: backslashes are literal

name = "Alice"
greeting = f"Hello, {name}!"           # f-string: embed variables directly
```



Note:

f-strings (formatted string literals) are the modern way to build strings with variables.

Common string operations

Operation	Example	Result
<code>+</code>	<code>"Hello" + " World"</code>	<code>"Hello World"</code>
<code>*</code>	<code>"ha" * 3</code>	<code>"hahaha"</code>
<code>.lower()</code>	<code>"HELLO".lower()</code>	<code>"hello"</code>
<code>.upper()</code>	<code>"hello".upper()</code>	<code>"HELLO"</code>
<code>.replace()</code>	<code>"hello".replace("h", "j")</code>	<code>"jello"</code>
<code>.strip()</code>	<code>" hello ".strip()</code>	<code>"hello"</code>
<code>.split()</code>	<code>"a,b,c".split(",")</code>	<code>["a", "b", "c"]</code>
<code>len()</code>	<code>len("hello")</code>	<code>5</code>

To apply the operations with a leading `.` just append the operation to the string, for example, `alice.upper()` gives `"ALICE"`. You can chain multiple operations together, e.g., `alice.upper().lower()`.

MiniTask: String operations on Alice in Wonderland

Copy from moodle the text in Minitask 1:

1. Paste everything into your Python terminal / IDE / or in an online Python editor e.g., **brython**
2. Strip the leading and trailing whitespace
3. Convert the whole text to lowercase
4. Count how many characters the cleaned text has using `len()`
5. Make "alice" start with a capital A using `.replace()`
6. Build a sentence using an f-string: "The passage has X characters." where X is the output of `len()`

Control Flow

Python uses **indentation** to define blocks of code — there are no curly braces {}.

```
if temperature > 30:
    print("It's hot!")    # indented → belongs to the if-block
    print("Stay cool.")  # also in the block
print("Done.")           # not indented → always runs
```

Python uses **indentation** to define blocks of code — there are no curly braces {}.

```
if temperature > 30:
    print("It's hot!")    # indented → belongs to the if-block
    print("Stay cool.")  # also in the block
print("Done.")           # not indented → always runs
```

Rules:

- Use **4 spaces** per level (the convention — your editor will handle this)
- Tabs and spaces must not be mixed — Python will raise an `IndentationError`
- Every block that follows a `:` must be indented

Indentation is **not** a style choice — it is syntax. Wrong indentation changes the meaning:

```
# Version A – message only prints when it's hot
```

```
if temperature > 30:  
    print("It's hot!")  
    print("Drink water.")
```

```
# Version B – message always prints
```

```
if temperature > 30:  
    print("It's hot!")  
print("Drink water.")
```

In most editors (VS Code, JupyterLab), pressing Tab inserts 4 spaces automatically. If you see an `IndentationError` or `TabError`, check for mixed tabs and spaces.

```
temperature: int = 28 # type hint: temperature is expected to be an integer. This
is optional but helps readability and AI understanding. You can omit it and just
write `temperature = 28`
```

```
if temperature > 30:
    print("It's hot!")
elif temperature > 20:
    print("Nice weather.")
else:
    print("A bit cold.")
```

Key rules:

- The colon `:` ends the condition
- The **indented block** below is what runs — Python uses indentation, not braces
- `elif` = “else if” — you can have as many as you need
- `else` is optional

Key rules:

- The colon `:` ends the condition
- The **indented block** below is what runs — Python uses indentation, not braces
- `elif` = “else if” — you can have as many as you need
- `else` is optional



Note:

Indentation is not a style choice in Python — it is syntax! Inconsistent indentation causes errors.

Indentation can be spaces or tabs, but the convention is to use 4 spaces. Most editors will handle/show this for you.

Conditions can be combined with and, or, not:

```
age: float = 22
has_ticket = True

if age >= 18 and has_ticket:
    print("Entry allowed.")

if not has_ticket:
    print("You need a ticket.")
```


When combining conditions, it helps to know the exact rules:

- and $\rightarrow$ True only if **both** conditions are True
- or $\rightarrow$ True if **at least one** condition is True
- not $\rightarrow$ flips the value: not True $\rightarrow$ False

Condition A	Condition B	A and B	A or B
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

MiniTask: Conditionals

Write a short Python script that:

1. Stores a temperature value in a variable
2. Prints "Freeze warning" if below 0, "Hot day" if above 35, and "All good" otherwise
3. Test it with at least three different values

We'll look at this together and I'll show you how each AI model (A, B, C) approaches it.

Exceptions

What are exceptions?

When Python encounters an error at runtime, it **raises an exception** — and stops before things go further wrong.

```
10 / 0           # ZeroDivisionError
int("hello")     # ValueError
x + 1            # NameError (if x was never defined)
"2" + 2          # TypeError
```

What are exceptions?

When Python encounters an error at runtime, it **raises an exception** — and stops before things go further wrong.

```
10 / 0           # ZeroDivisionError
int("hello")     # ValueError
x + 1            # NameError (if x was never defined)
"2" + 2          # TypeError
```

Exceptions are **not your enemy**:

- They stop the program before it goes haywire
- The **traceback** tells you exactly where and why it failed
- The earlier an error is caught, the cheaper it is to fix

Handling exceptions with `try / except`

Wrap code that might fail in a `try` block. Handle the failure in `except`:

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Can't divide by zero!")
```


Handling exceptions with `try / except`

Wrap code that might fail in a `try` block. Handle the failure in `except`:

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Can't divide by zero!")
```

You can catch multiple types, add an `else` (runs if no error), and a `finally` (runs no matter what):


```
try:
    value = int(input("Enter a number: "))
except ValueError:
    print("That's not a number.")
else:
    print(f"You entered: {value}")
finally:
    print("Done.")    # always runs
```

Common exception types

Exception	When it occurs	Example
ZeroDivisionError	dividing by zero	<code>1 / 0</code>
TypeError	wrong type for an operation	<code>"2" + 2</code>
ValueError	right type, bad value	<code>int("hello")</code>
NameError	variable not defined	<code>print(x)</code>
IndexError	index out of range	<code>[1,2,3][5]</code>
KeyError	key not found in a dict	<code>d["missing"]</code>
FileNotFoundError	file doesn't exist	<code>open("x.txt")</code>



Note:

You don't need to memorise these — just recognise them when they appear. AI is also very good at explaining tracebacks.

MiniTask: Try / except

The following code crashes on some inputs. Wrap it in a `try/except` to handle the errors gracefully.

```
numbers = ["3.14", "2.71", "not a number", "1.41", "0"]

for item in numbers:
    result = 1 / float(item)
    print(f"1 / {item} = {result:.4f}")
```

1. Run it as-is and read the traceback — which line fails and why?
2. Wrap the risky lines in `try/except` so the loop continues on bad values
3. Print a helpful message when a value can't be converted or causes a division error
4. *Bonus:* catch `ValueError` and `ZeroDivisionError` separately with different messages

LBYL vs EAFP

There are two schools of thought on handling errors in Python:

Look Before You Leap (LBYL)

```
if age >= 18:  
    buy_ticket()
```

Easier to Ask Forgiveness than Permission (EAFP)

```
try:  
    buy_ticket()  
except AgeError:  
    print("Too young!")
```

Python culture leans towards **EAFP** — it's faster and more readable when exceptions are rare.

Loops

Lists — a preview

Before we look at loops, you need to know one data structure: the **list**.

```
fruits = ["apple", "banana", "cherry"]  
depths = [12.4, 15.1, 9.8, 21.3]  
mixed = [1, "two", 3.0, True]    # lists can hold any type
```


Lists — a preview

Before we look at loops, you need to know one data structure: the **list**.

```
fruits = ["apple", "banana", "cherry"]
depths = [12.4, 15.1, 9.8, 21.3]
mixed  = [1, "two", 3.0, True]      # lists can hold any type
```

Lists are **ordered, mutable** sequences of items. You access items by index:

```
fruits[0]      # "apple"    – indexing starts at 0
fruits[-1]     # "cherry"   – negative index counts from the end
len(fruits)    # 3
```

Note the 0 and square brackets [] syntax!



Note:

Lists, dictionaries, tuples, and sets are Python's main data structures. We'll cover all of them properly in the next lecture.

A for loop iterates over a **sequence** — a list, a string, a range of numbers.

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:  
    print(fruit)
```

```
# apple  
# banana  
# cherry
```

Using `range()` to loop over numbers:

```
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8):   # 2, 3, 4, 5, 6, 7
    print(i)
```

enumerate — loop with an index

When you need both the **item** and its **position**, use `enumerate`:

```
fruits = ["apple", "banana", "cherry"]

for index, fruit in enumerate(fruits):
    print(f'The index is {index} and the fruit is {fruit}')

# The index is 0 and the fruit is apple
# The index is 1 and the fruit is banana
# The index is 2 and the fruit is cherry
```

while loops — a brief mention

A while loop runs **as long as a condition is true**. Use it when you don't know in advance how many iterations you need.

```
count = 0
while count < 5:
    print(count)
    count += 1 # same as count = count + 1
```

What will this code do?

```
count = 1
while count > 0:
    print(count)
    count += 1
```

MiniTask: Loops

Start from your cleaned Alice text (lowercase, stripped). Split it into sentences on "." and loop over them.

1. Use `.split(".")` to turn the text into a list of sentences
2. Write a for loop that prints each sentence on a new line
3. Modify it to use `enumerate` so each line is numbered (starting from 1)
4. Inside the loop, call `.capitalize()` on each sentence before printing

Expected output:

```
1: Alice was beginning to get very tired of sitting by her sister on the
bank
2: And of having nothing to do: once or twice she had peeped into the book
her sister was reading
3: But it had no pictures or conversations in it
...
```

I'll show the solution using Model B — writing it live with AI as a pair programmer.

MiniTask: Read and explain

Your AI generated the following script. Read it carefully and answer the questions below.

I'll show the solution using Model A — asking AI to explain the code.

```
samples = ["granite", "basalt", "limestone", "sandstone", "shale"]
long_names = []

for i, rock in enumerate(samples):
    if len(rock) > 7:
        long_names.append(rock)
        print(f"Sample {i}: '{rock}' has a long name")

print(f"Found {len(long_names)} long names: {long_names}")
```

1. What does `len(rock)` return?
2. What does `long_names.append(rock)` do?
3. What would the output be?

Recap

After today, you should be able to:

- **Explain** why understanding code matters more than memorising syntax in the AI era
- **Recognise** three ways to use AI while coding: explainer, pair programmer, starting point
- **Name** Python's basic types: `int`, `float`, `complex`, `str`, `bool`, `None`
- **Apply** PEP 8 naming conventions: `snake_case`, type hints, `UPPER_CASE` constants
- **Write** conditional statements with `if`, `elif`, `else`, and logical operators
- **Write** for loops over lists and ranges, including with `enumerate`
- **Read** a short Python script and explain what it does